MIS41110 Practical Sheet, Week 4

Covers the Week 4 lecture (lists, tuples, sets, dictionaries, comprehensions, text scoring) · **Released** with the Week 4 slides · **Worked** Monday of Week 5 · **Solutions** that afternoon

Solutions from your tutors after the day's last session.

Monday 5 October: a strike from 12:00 to 13:00 reshapes the practical. *For the tutorial, come to the 10:00–12:00 or the 15:00–17:00 session. The middle session moves to 13:00–14:30 and is for questions only: the tutorial does not run in it, but the tutors are there to answer your questions.*

Exercise 1: colony counts, listed

Start from `counts = [242, 60, 120, 80, 182]` (five census entries).

1. Print the first entry, the last entry (negative index), and the middle three (one slice).
2. A sixth census arrives: append `140`.
3. The `60` was a miscount; **replace it in place** with `74` (assign at the index; no rebuilding the list).
4. Print the total, the largest entry, and the list sorted (keep the original list unsorted; print a sorted *copy*). Three honest builtins to take as given: `max()`, `min()`, and `sorted()` do exactly what they say.
5. How many times does `242` appear (`count`)? Then `pop()` the last entry and print what came off.

Exercise 2: tag the species

Each census line is a **tuple** `(species, count)`. Build the list

```
census = [("adelie", 242), ("emperor", 60), ("gentoo", 120), ("chinstrap", 80)]
```

1. Loop over it and print `adelie: 242` style lines, unpacking each tuple in the `for` line itself.
2. Swap two variables `a, b = b, a` and explain, in a comment, what tuples have to do with it.
3. Try `census[0][1] = 250`. Read the error; say in a comment why tuples refuse.

Exercise 3: who has been out on the ice?

Two day rosters:

```
monday = {"vera", "tom", "ines", "kofi"}
tuesday = {"tom", "kofi", "lena"}
```

Print, each from one set expression: everyone who went out at least once; those who went out both days; those who went out Monday only; those who went out exactly one of the two days.

Exercise 4: the supply ledger

Start from `supplies = {"fuel drum": 17, "ration box": 42, "med kit": 9}`.

1. A crate of `"tea chest"` arrives: add it with count `12`.
2. Ten ration boxes go out to a field camp: update the count.
3. Print all item names, then all counts, then the total number of items on station (`sum` over the values).
4. Build, with a **dict comprehension**, a dict of the same items where every count is doubled (the resupply plan), and print it.

Exercise 5: the census grid

Rows are days (Monday, Tuesday, Wednesday), columns are colonies (North Ridge, Seal Point, Windward):

```
grid = [[242, 182, 210],
        [245, 185, 213],
        [240, 180, 208]]
```

1. Print the middle day's counts (*one row*), and Seal Point's Tuesday count (*one value*).
2. Compute each colony's three-day total with a loop over row indexes (column sums, matrix-style).
3. One comprehension: a list of each **day's** total.

Exercise 5b ☆: name the smallest colony (if the session runs short, finish at home)

Two lists, same length, one describing the other:

```
colonies = ["North Ridge", "Seal Point", "Windward"]
counts   = [242, 60, 120]
```

`min(counts)` tells you the smallest count. It does **not** tell you whose it is. Write `smallest_colony(colonies, counts)` that returns the **name** of the colony with the fewest penguins.

The trick is to track an **index**, not a value: start with `best = 0`, walk the positions with `for i in range(len(counts))`, and whenever `counts[i]` beats `counts[best]`, remember `i`. At the end, `colonies[best]` reaches across to the other list.

`smallest_colony(colonies, counts)` returns `'Seal Point'`. Check it still works when the smallest is first, and when it is last.

Exercise 6: the morale scorer (case study)

`data/field_notes.txt` holds `colony,note` lines (its first line is a **header**: read it off and ignore it); `data/morale_words.txt` holds `word,weight` lines. **Before any code**, write the plan as four pseudo-code lines, Week-3 style (load the lexicon → read the notes → score each note → accumulate per colony). Then build one line of the plan at a time:

1. loads the lexicon into a dict (`split(",")` per line, `float` the weight);
2. reads each note (`split(",", 1)`, so commas inside the note survive), scores it by summing the weights of known words, and accumulates a **total score per colony** in a dict;
3. prints one line per colony, the total rounded to one decimal:

```
North Ridge: 2.4
Seal Point: 3.8
Windward: 1.7
```

(a) The note `"generator broken again, crew tired and hungry"` scores `-1.7`. Recompute that by hand from the lexicon to check your loop.

(b) *Optional extension, best finished at home:* pick five notes and label them yourself, good day or bad day. Build a mini lexicon from your labels alone: every word in a good note gets `+1`, every word in a bad note gets `-1`. Score all twenty notes with it and compare the colony ranking against `morale_words.txt`. (Labels are your call; that judgment is the whole lesson.)

Pick the structure that fits the question: order → list, fixed shape → tuple, uniqueness → set, lookup → dict.